Experiment – 1

Comparative Study of Low-Code Development

versus Traditional Application Development

AIM

This experiment sets out to contrast the low-code and conventional ways of building applications, looking at how each one is carried out, how its workflow runs, and which technical strengths and drawbacks come with it.

Introduction to Traditional Approach

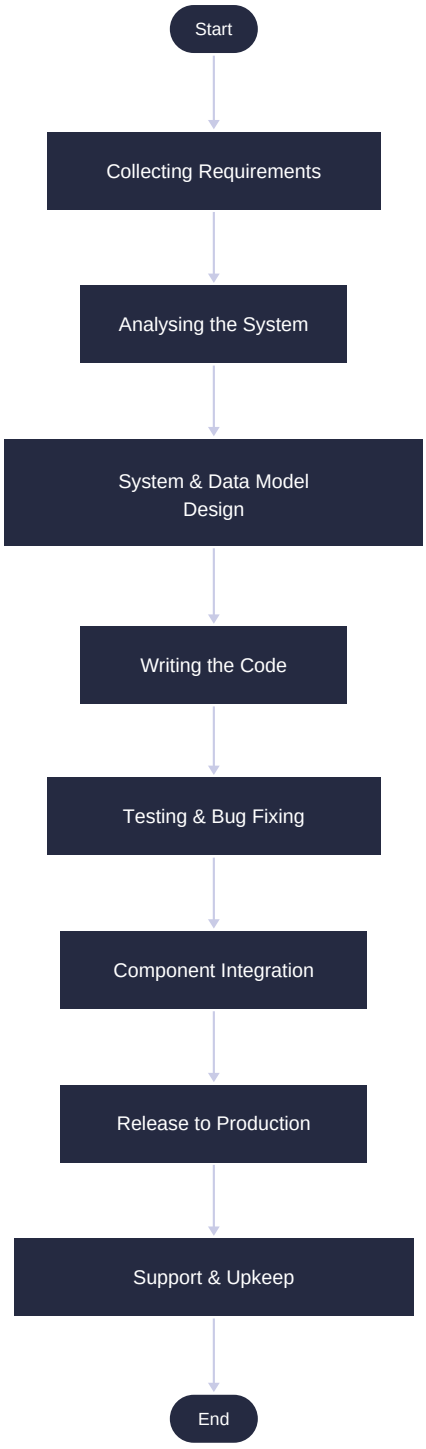
In conventional development, an application is created by hand-writing source code in a programming language, with the help of IDEs, compilers and other supporting utilities. The development team takes charge of designing the database, coding the business rules, connecting external APIs, and later releasing and looking after the finished product. Since everything is written from scratch, teams gain a great deal of control over the architecture and behaviour of the software and can fit it closely to what the client needs. The price of this freedom is longer delivery cycles, a need for experienced engineers, and a heavy testing effort to keep the system dependable and fast.

Introduction to Low Code Approach

Low-code platforms offer a different route to building software, one that cuts down on the amount of hand-written code. Rather than coding every feature, a builder assembles the application on a visual canvas using ready-made components and automation that the platform already supplies. As a result, apps can be created and released in far less time, and the platform can be used by seasoned programmers and by people with little programming background alike. Although this saves both time and money, the approach can feel restrictive when an application needs very specialised behaviour or deeper control over how it works.

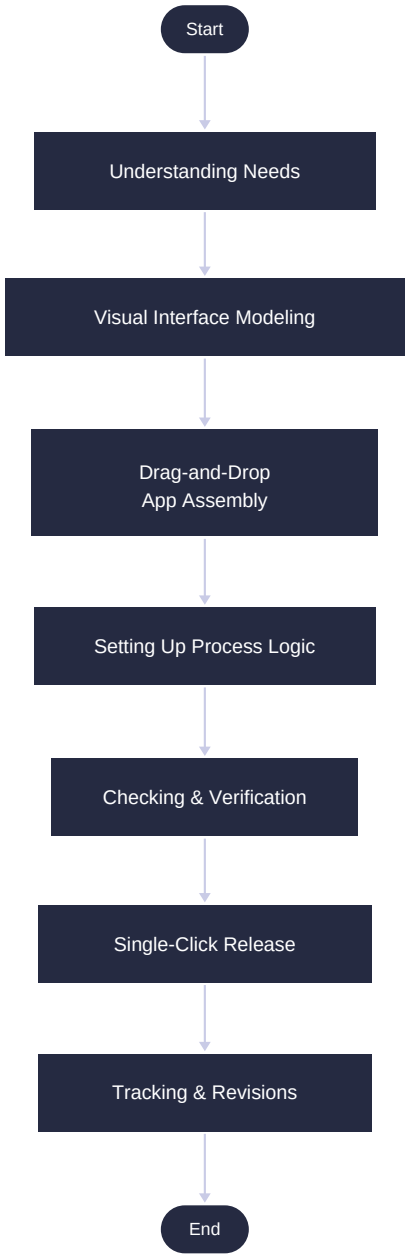
Stages in Traditional Approach

The classic software development life cycle follows an orderly, engineering-driven series of phases, and each phase demands its own dedicated effort:



Stages in Low Code Approach

In the low-code workflow, hand-coding phases give way to steps carried out through configuration, and each of these calls for much less direct engineering work:



Comparative Analysis

Aspect	Conventional Coding	Low-Code Platform
Build Time	Longer, because each module has to be coded by hand and tested one at a time	Much shorter, since reusable prebuilt blocks greatly cut the effort of building

Expertise Needed	Calls for experienced programmers who know the languages and frameworks well	Very little coding knowledge is enough; visual tools let business users build apps
Flexibility	Complete freedom: any behaviour can be coded exactly as wanted	Limited to the components and connectors the platform supplies
Expense	Higher initial outlay for developer effort and infrastructure	Cheaper to begin with, but recurring subscription or license charges follow
Upkeep & Upgrades	Each bug fix or new feature means editing code by hand and retesting	The vendor rolls out upgrades centrally, so little manual work remains
Platform Dependence	None: the organization fully owns and controls its source code	Yes: the app stays bound to the vendor's platform and ecosystem
Ideal Scenarios	Large, high-performance, or security-critical systems	Quick prototypes, in-house tools, and process automation

Results

The comparison shows that conventional development fits best where an application needs deep customization, top performance, or compliance with strict industry regulations, because it gives full control over the design and behaviour of the system. Low-code tools, on the other hand, are a sensible pick when business applications must be delivered quickly and at low cost. For this reason, many organizations follow a blended strategy: low-code for rapidly building interfaces and workflows, and hand-written code for intricate backend logic and special requirements.